



Matplotlib – Essential Cheat Sheet

For Data Visualization in Python

1. Basic Setup

```
import matplotlib.pyplot as plt
import numpy as np

# Optional: Better default styles
plt.style.use('ggplot')          # alternatives: 'seaborn', 'fivethirtyeight',
                                  'bmh'
# plt.style.available → view all styles

# Global settings
plt.rcParams['figure.figsize'] = (8, 5)
plt.rcParams['font.size'] = 12
plt.rcParams['figure.dpi'] = 100
```

2. Most Common Plot Types

```
x = np.linspace(0, 10, 200)
y = np.sin(x)
z = np.cos(x)
```

1. Line Plot

```
plt.plot(x, y, label='sin(x)', color='blue', linewidth=2)
plt.plot(x, z, linestyle='--', color='red', label='cos(x)')
```

2. Scatter Plot

```
plt.scatter(x, y, s=15, c='purple', alpha=0.6, edgecolor='black')
```

3. Bar Plot

```
categories = ['A', 'B', 'C', 'D']
values = [45, 72, 19, 88]
```

```
plt.bar(categories, values, color='#44aa77')
```

4. Histogram

```
data = np.random.randn(10000)
plt.hist(data, bins=50, color='teal', edgecolor='black', alpha=0.7)
```

5. Box Plot

```
plt.boxplot([
    np.random.normal(0,1,200),
    np.random.normal(2,1.5,200)
], labels=['A', 'B'])
```

6. Pie Chart (less commonly used today)

```
plt.pie([30, 20, 25, 25],
        labels=['Python', 'JS', 'Java', 'C++'],
        autopct='%1.0f%%')
```

3. Important Customization

1. Figure

```
plt.figure(figsize=(10,6), dpi=120)
```

2. More on plot

```
plt.plot(x, y, 'o--',
         color='purple',
         lw=2,
         markersize=8,
         markerfacecolor='yellow')
```

3. Title

```
plt.title("Main Title", fontsize=16, fontweight='bold', pad=15)
```

```
plt.suptitle("Super Title", fontsize=20, y=1.02)
```

4. Labels

```
plt.xlabel("X Axis → Time (s)", fontsize=13)  
plt.ylabel("Y Axis → Amplitude", fontsize=13)
```

5. Background

```
plt.grid(True, linestyle='--', alpha=0.6)  
plt.legend(loc='upper right', fontsize=11, frameon=True, shadow=True)
```

6. Axis limits

```
plt.xlim(0, 10)  
plt.ylim(-1.5, 1.5)
```

7. Tick customization

```
plt.xticks(np.arange(0, 11, 2), rotation=45)  
plt.yticks([-1, 0, 1], ['-ve', 'zero', '+ve'])
```

4. Multiple Plots (Subplots – Very Important)

Method 1 – Recommended (Modern)

```
fig, axes = plt.subplots(nrows=2, ncols=2,  
                        figsize=(12,8),  
                        sharex=True)  
  
axes[0,0].plot(x, np.sin(x), 'b')  
axes[0,0].set_title('sin')  
  
axes[0,1].plot(x, np.cos(x), 'r')  
axes[0,1].set_title('cos')  
  
axes[1,0].plot(x, np.tan(x), 'g')  
axes[1,0].set_title('tan')  
  
fig.suptitle("Four Plots", fontsize=18, y=1.02)
```

```
plt.tight_layout()    # Fix spacing automatically
```

Method 2 – Legacy Style

```
plt.subplot(2,2,1)
plt.plot(x, y)

plt.subplot(2,2,2)
plt.plot(x, z)
```

5. Advanced / Frequently Used Features

Twin axes (two Y-axes)

```
fig, ax1 = plt.subplots()

ax1.plot(x, y, 'b-', label='sin')
ax1.set_ylabel('sin', color='b')

ax2 = ax1.twinx()
ax2.plot(x, z*100, 'r-', label='cos x 100')
ax2.set_ylabel('cos scaled', color='r')
```

Annotation

```
plt.annotate('Peak!',
             xy=(np.pi/2, 1),
             xytext=(5, 0.6),
             arrowprops=dict(facecolor='black', shrink=0.05))
```

Fill between curves

```
plt.fill_between(x, y, z,
                 where=(y > z),
                 color='gray',
                 alpha=0.3,
                 interpolate=True)
```

Save figure

```
plt.savefig('myplot.png', dpi=300, bbox_inches='tight')  
# plt.savefig('myplot.pdf', bbox_inches='tight') # vector format
```

6. Quick One-Liners

```
plt.plot(x, y); plt.show()  
  
plt.plot(x, y, 'r.-')  
plt.grid()  
plt.show()  
  
plt.hist(np.random.randn(5000), bins=60, density=True)
```

7. Popular Styles (2024–2026)

```
plt.style.use('seaborn-v0_8-whitegrid')  
plt.style.use('ggplot')  
plt.style.use('fivethirtyeight')  
plt.style.use('bmh')  
plt.style.use('dark_background')
```

Summary – Must Remember Commands

```
plt.plot() / plt.scatter() / plt.bar() / plt.hist()  
  
plt.title() / plt.xlabel() / plt.ylabel() / plt.legend()  
  
plt.grid() / plt.xlim() / plt.ylim() / plt.xticks() / plt.yticks()  
  
fig, ax = plt.subplots()  
  
plt.tight_layout()  
  
plt.savefig(..., dpi=300, bbox_inches='tight')
```
